

Vector

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Vector< T > Class Template Reference	5
3.1.1 Detailed Description	8
3.1.2 Member Typedef Documentation	8
3.1.2.1 allocator_type	8
3.1.2.2 const_iterator	8
3.1.2.3 const_pointer	8
3.1.2.4 const_reference	9
3.1.2.5 const_reverse_iterator	9
3.1.2.6 difference_type	9
3.1.2.7 iterator	9
3.1.2.8 pointer	9
3.1.2.9 reference	9
3.1.2.10 reverse_iterator	9
3.1.2.11 size_type	10
3.1.2.12 value_type	10
3.1.3 Constructor & Destructor Documentation	10
3.1.3.1 Vector() [1/5]	10
3.1.3.2 Vector() [2/5]	11
3.1.3.3 Vector() [3/5]	12
3.1.3.4 Vector() [4/5]	12
3.1.3.5 Vector() [5/5]	13
3.1.3.6 ~Vector()	13
3.1.4 Member Function Documentation	13
3.1.4.1 append_range()	13
3.1.4.2 assign() [1/3]	14
3.1.4.3 assign() [2/3]	14
3.1.4.4 assign() [3/3]	14
3.1.4.5 assign_range()	14
3.1.4.6 at()	15
3.1.4.7 back()	15
3.1.4.8 begin() [1/2]	15
3.1.4.9 begin() [2/2]	15
3.1.4.10 capacity()	16
3.1.4.11 cbegin()	17
3.1.4.12 cend()	17
3.1.4.13 clear()	17

3.1.4.14 <code>crbegin()</code>	18
3.1.4.15 <code>crend()</code>	18
3.1.4.16 <code>data()</code>	18
3.1.4.17 <code>emplace()</code>	19
3.1.4.18 <code>emplace_back()</code>	19
3.1.4.19 <code>empty()</code>	20
3.1.4.20 <code>end()</code> [1/2]	20
3.1.4.21 <code>end()</code> [2/2]	20
3.1.4.22 <code>erase()</code> [1/3]	21
3.1.4.23 <code>erase()</code> [2/3]	21
3.1.4.24 <code>erase()</code> [3/3]	22
3.1.4.25 <code>erase_if()</code>	22
3.1.4.26 <code>front()</code>	23
3.1.4.27 <code>get_allocator()</code>	23
3.1.4.28 <code>insert()</code> [1/5]	24
3.1.4.29 <code>insert()</code> [2/5]	24
3.1.4.30 <code>insert()</code> [3/5]	25
3.1.4.31 <code>insert()</code> [4/5]	25
3.1.4.32 <code>insert()</code> [5/5]	26
3.1.4.33 <code>insert_range()</code>	26
3.1.4.34 <code>max_size()</code>	27
3.1.4.35 <code>operator!=(())</code>	27
3.1.4.36 <code>operator<()</code>	27
3.1.4.37 <code>operator<=()</code>	28
3.1.4.38 <code>operator=()</code> [1/2]	28
3.1.4.39 <code>operator=()</code> [2/2]	29
3.1.4.40 <code>operator==(())</code>	29
3.1.4.41 <code>operator>()</code>	30
3.1.4.42 <code>operator>=()</code>	30
3.1.4.43 <code>operator[]()</code>	30
3.1.4.44 <code>pop_back()</code>	31
3.1.4.45 <code>push_back()</code> [1/2]	31
3.1.4.46 <code>push_back()</code> [2/2]	32
3.1.4.47 <code>rbegin()</code> [1/2]	32
3.1.4.48 <code>rbegin()</code> [2/2]	33
3.1.4.49 <code>rend()</code> [1/2]	33
3.1.4.50 <code>rend()</code> [2/2]	33
3.1.4.51 <code>reserve()</code>	34
3.1.4.52 <code>resize()</code> [1/2]	35
3.1.4.53 <code>resize()</code> [2/2]	35
3.1.4.54 <code>shrink_to_fit()</code>	35
3.1.4.55 <code>size()</code>	36

3.1.4.56 swap() [1/2]	36
3.1.4.57 swap() [2/2]	36
4 File Documentation	39
4.1 vector.h File Reference	39
4.2 vector.h	40
4.3 vector.hpp File Reference	41
4.3.1 Function Documentation	42
4.3.1.1 erase()	42
4.3.1.2 erase_if()	43
4.3.1.3 swap()	43
4.4 vector.hpp	43
Index	51

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Vector< T >	5
-----------------------------------	---

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

vector.h		39
vector.hpp		41

Chapter 3

Class Documentation

3.1 Vector< T > Class Template Reference

```
#include <vector.h>
```

Public Types

- using `value_type` = T
- using `allocator_type` = std::allocator<T>
- using `size_type` = std::size_t
- using `difference_type` = std::ptrdiff_t
- using `reference` = value_type&
- using `const_reference` = const value_type&
- using `pointer` = value_type*
- using `const_pointer` = const value_type*
- using `iterator` = value_type*
- using `const_iterator` = const value_type*
- using `reverse_iterator` = std::reverse_iterator<iterator>
- using `const_reverse_iterator` = std::reverse_iterator<const_iterator>

Public Member Functions

- `Vector` ()
constructors, destructor, assignment operators
- `Vector` (size_type s)
konstruktorius
- `Vector` (std::initializer_list< T > init)
konstruktorius su dydziu
- `Vector` (const `Vector` &r)
konstruktorius su initializer list
- `Vector` & `operator=` (const `Vector` &r)
kopijavimo konstruktorius
- `Vector` (`Vector` &&r)
kopijavimo priskyrimo operatorius
- `Vector` & `operator=` (`Vector` &&r)

- perkelimo(move) konstruktorius*
- [~Vector](#) ()
 - perkelimo(move) priskyrimo operatorius*
- void [assign](#) (size_type count, const T &value)
 - destruktorius*
- void [assign](#) (std::initializer_list< T > ilist)
 - assign funkcija su initializer list*
- template<typename InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>>
 void [assign](#) (InputIt first, InputIt last)
 - assign funkcija su iteratoriais(range)*
- template<typename R>
 constexpr void [assign_range](#) (R &&r)
 - assign funkcija su range*
- [allocator_type](#) [get_allocator](#) () const
 - get allocator funkcija*
- [reference](#) at (size_type pos)
 - element access*
- [reference](#) operator[] (size_type pos)
 - operator []*
- [reference](#) front ()
 - front funkcija*
- [reference](#) back ()
 - back funkcija*
- T * [data](#) () noexcept
 - data funkcija*
- [iterator](#) begin () noexcept
 - iterators*
- [const_iterator](#) begin () const noexcept
- [const_iterator](#) cbegin () const noexcept
 - cbegin funkcija*
- [iterator](#) end () noexcept
 - end funkcija*
- [const_iterator](#) end () const noexcept
 - const end funkcija*
- [const_iterator](#) cend () const noexcept
 - cend funkcija*
- [reverse_iterator](#) rbegin () noexcept
 - rbegin funkcija*
- [const_reverse_iterator](#) rbegin () const noexcept
 - const rbegin funkcija*
- [const_reverse_iterator](#) crbegin () const noexcept
 - crbegin funkcija*
- [reverse_iterator](#) rend () noexcept
 - rend funkcija*
- [const_reverse_iterator](#) rend () const noexcept
 - const rend funkcija*
- [const_reverse_iterator](#) crend () const noexcept
 - crend funkcija*
- bool [empty](#) () const noexcept
 - capacity*
- size_type [size](#) () const noexcept

- size funkcija*
- `size_type max_size ()` const noexcept
max size funkcija
- `size_type capacity ()` const noexcept
capacity funkcija
- void `reserve (size_type new_cap)`
reserve funkcija
- void `shrink_to_fit ()`
shrink to fit funkcija
- void `clear ()` noexcept
modifiers
- `iterator insert (const_iterator pos, const T &value)`
insert funkcija su const T& value
- `iterator insert (const_iterator pos, T &&value)`
insert funkcija su T&& value
- `iterator insert (const_iterator pos, size_type count, const T &value)`
insert funkcija su count ir value
- `template<typename InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>`
`iterator insert (const_iterator pos, InputIt first, InputIt last)`
insert funkcija su iteratoriais
- `iterator insert (const_iterator pos, std::initializer_list< T > ilist)`
insert funkcija su initializer list
- `constexpr iterator insert_range (const_iterator pos, std::initializer_list< T > ilist)`
insert range funkcija su initializer list
- `template<class... Args>`
`iterator emplace (const_iterator pos, Args &&... args)`
emplace funkcija
- `template<typename... Args>`
`reference emplace_back (Args &&... args)`
emplace back funkcija
- `iterator erase (const_iterator pos)`
erase funkcija su vienu iteratoriumi
- `iterator erase (const_iterator first, const_iterator last)`
erase funkcija intervalui
- void `push_back (const T &value)`
push back funkcija su const T& value
- void `push_back (T &&value)`
push back funkcija su T&& value
- `template<typename R>`
`constexpr void append_range (R &&rg)`
append range funkcija
- void `pop_back ()`
pop back funkcija
- void `resize (size_type count)`
resize funkcija su vienu parametru
- void `resize (size_type count, const value_type &value)`
resize funkcija su dviem parametrais
- `constexpr void swap (Vector &other)` noexcept
swap funkcija
- bool `operator== (const Vector &other)` const
non-member functions

- bool `operator!=` (const `Vector` &other) const
operator !=
- bool `operator<` (const `Vector` &other) const
operator <
- bool `operator<=` (const `Vector` &other) const
operator <=
- bool `operator>` (const `Vector` &other) const
operator >
- bool `operator>=` (const `Vector` &other) const
operator >=
- void `swap` (`Vector` &a, `Vector` &b) noexcept
- void `erase` (`Vector` &v, const T &value)
- template<typename Pred>
void `erase_if` (`Vector` v, Pred p)

3.1.1 Detailed Description

```
template<typename T>
class Vector< T >
```

Definition at line 9 of file [vector.h](#).

3.1.2 Member Typedef Documentation

3.1.2.1 allocator_type

```
template<typename T>
using Vector< T >::allocator_type = std::allocator<T>
```

Definition at line 13 of file [vector.h](#).

3.1.2.2 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = const value_type*
```

Definition at line 21 of file [vector.h](#).

3.1.2.3 const_pointer

```
template<typename T>
using Vector< T >::const_pointer = const value_type*
```

Definition at line 19 of file [vector.h](#).

3.1.2.4 const_reference

```
template<typename T>
using Vector< T >::const_reference = const value_type&
```

Definition at line 17 of file [vector.h](#).

3.1.2.5 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Definition at line 23 of file [vector.h](#).

3.1.2.6 difference_type

```
template<typename T>
using Vector< T >::difference_type = std::ptrdiff_t
```

Definition at line 15 of file [vector.h](#).

3.1.2.7 iterator

```
template<typename T>
using Vector< T >::iterator = value_type*
```

Definition at line 20 of file [vector.h](#).

3.1.2.8 pointer

```
template<typename T>
using Vector< T >::pointer = value_type*
```

Definition at line 18 of file [vector.h](#).

3.1.2.9 reference

```
template<typename T>
using Vector< T >::reference = value_type&
```

Definition at line 16 of file [vector.h](#).

3.1.2.10 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

Definition at line 22 of file [vector.h](#).

3.1.2.11 size_type

```
template<typename T>  
using Vector< T >::size_type = std::size_t
```

Definition at line 14 of file [vector.h](#).

3.1.2.12 value_type

```
template<typename T>  
using Vector< T >::value_type = T
```

Definition at line 12 of file [vector.h](#).

3.1.3 Constructor & Destructor Documentation

3.1.3.1 Vector() [1/5]

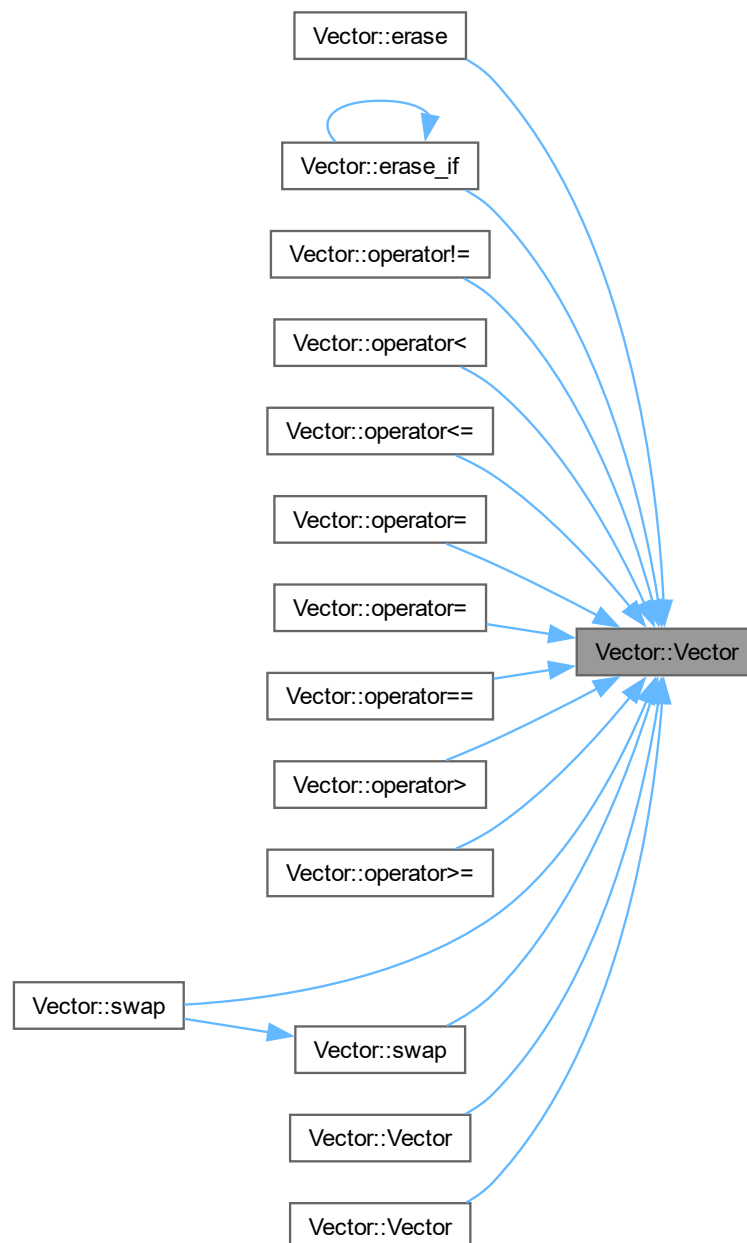
```
template<typename T>  
Vector< T >::Vector ()
```

constructors, destructor, assignment operators

defaultinis konstruktorius

Definition at line 4 of file [vector.hpp](#).

Here is the caller graph for this function:



3.1.3.2 Vector() [2/5]

```
template<typename T>
Vector< T >::Vector (
    size_type s)
```

konstruktorius

konstruktorius su dydziu

Definition at line 11 of file [vector.hpp](#).

3.1.3.3 Vector() [3/5]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > init)
```

konstruktorius su dydziu

konstruktorius su initializer list

Definition at line 18 of file [vector.hpp](#).

3.1.3.4 Vector() [4/5]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & r)
```

konstruktorius su initializer list

kopijavimo konstruktorius

Definition at line 36 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.3.5 Vector() [5/5]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && r)
```

kopijavimo priskyrimo operatorius

perkelimo(move) konstruktorius

Definition at line 46 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.3.6 ~Vector()

```
template<typename T>
Vector< T >::~~Vector ()
```

perkelimo(move) priskyrimo operatorius

Definition at line 29 of file [vector.hpp](#).

3.1.4 Member Function Documentation

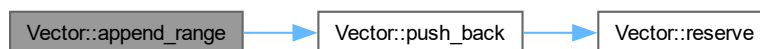
3.1.4.1 append_range()

```
template<typename T>
template<typename R>
void Vector< T >::append_range (
    R && rg) [constexpr]
```

append range funkcija

Definition at line 475 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.2 assign() [1/3]

```
template<typename T>
template<typename InputIt, typename>
void Vector< T >::assign (
    InputIt first,
    InputIt last)
```

assign funkcija su iteratoriais(range)

Definition at line 111 of file [vector.hpp](#).

3.1.4.3 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    size_type count,
    const T & value)
```

destruktorius

assign funkcija su dviem parametrais

member functions

Definition at line 87 of file [vector.hpp](#).

3.1.4.4 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > ilist)
```

assign funkcija su initializer list

jei cap yra 0, tai elem bus nullptr, kitaip bus sukuriamas naujas masyvas

Definition at line 98 of file [vector.hpp](#).

3.1.4.5 assign_range()

```
template<typename T>
template<typename R>
void Vector< T >::assign_range (
    R && r) [constexpr]
```

assign funkcija su range

Definition at line 124 of file [vector.hpp](#).

3.1.4.6 at()

```
template<typename T>
Vector< T >::reference Vector< T >::at (
    size_type pos)
```

element access

at funkcija

Definition at line 141 of file [vector.hpp](#).

3.1.4.7 back()

```
template<typename T>
Vector< T >::reference Vector< T >::back ()
```

back funkcija

Definition at line 159 of file [vector.hpp](#).

3.1.4.8 begin() [1/2]

```
template<typename T>
Vector< T >::const_iterator Vector< T >::begin () const [noexcept]
```

Definition at line 174 of file [vector.hpp](#).

3.1.4.9 begin() [2/2]

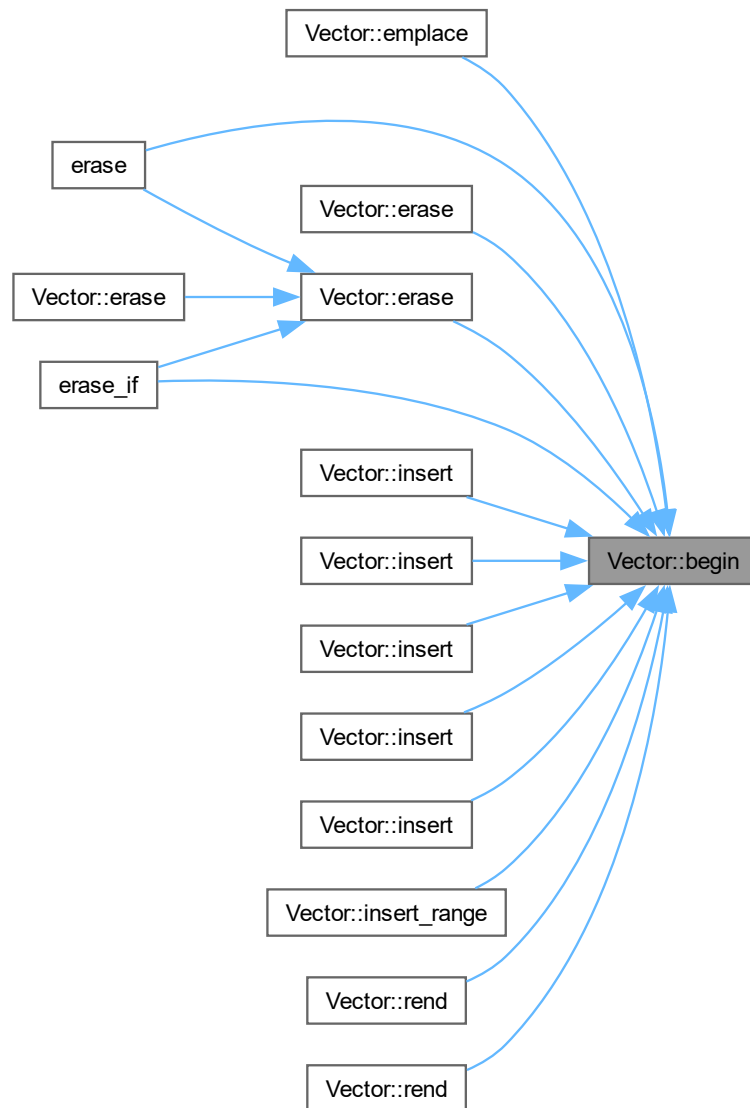
```
template<typename T>
Vector< T >::iterator Vector< T >::begin () [noexcept]
```

iterators

begin funkcija

Definition at line 169 of file [vector.hpp](#).

Here is the caller graph for this function:



3.1.4.10 capacity()

```
template<typename T>
Vector< T >::size_type Vector< T >::capacity () const [noexcept]
```

capacity funkcija

Definition at line 244 of file [vector.hpp](#).

3.1.4.11 cbegin()

```
template<typename T>
Vector< T >::const_iterator Vector< T >::cbegin () const [noexcept]
```

cbegin funkcija

Definition at line 179 of file [vector.hpp](#).

Here is the caller graph for this function:



3.1.4.12 cend()

```
template<typename T>
Vector< T >::const_iterator Vector< T >::cend () const [noexcept]
```

cend funkcija

Definition at line 194 of file [vector.hpp](#).

Here is the caller graph for this function:



3.1.4.13 clear()

```
template<typename T>
void Vector< T >::clear () [noexcept]
```

modifiers

clear funkcija

Definition at line 279 of file [vector.hpp](#).

3.1.4.14 crbegin()

```
template<typename T>  
Vector< T >::const_reverse_iterator Vector< T >::crbegin () const [noexcept]
```

crbegin funkcija

Definition at line 209 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.15 crend()

```
template<typename T>  
Vector< T >::const_reverse_iterator Vector< T >::crend () const [noexcept]
```

crend funkcija

Definition at line 224 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.16 data()

```
template<typename T>  
T * Vector< T >::data () [noexcept]
```

data funkcija

Definition at line 164 of file [vector.hpp](#).

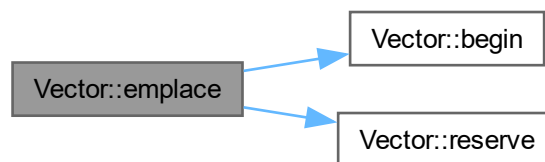
3.1.4.17 `emplace()`

```
template<typename T>
template<class... Args>
Vector< T >::iterator Vector< T >::emplace (
    const_iterator pos,
    Args &&... args)
```

`emplace` funkcija

Definition at line 403 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.18 `emplace_back()`

```
template<typename T>
template<typename... Args>
Vector< T >::reference Vector< T >::emplace_back (
    Args &&... args)
```

`emplace back` funkcija

Definition at line 420 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.19 empty()

```
template<typename T>  
bool Vector< T >::empty () const [noexcept]
```

capacity

empty funkcija

Definition at line 229 of file [vector.hpp](#).

3.1.4.20 end() [1/2]

```
template<typename T>  
Vector< T >::const_iterator Vector< T >::end () const [noexcept]
```

const end funkcija

Definition at line 189 of file [vector.hpp](#).

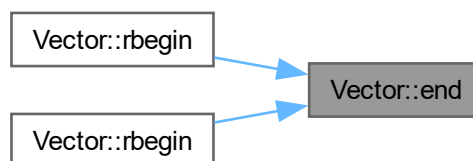
3.1.4.21 end() [2/2]

```
template<typename T>  
Vector< T >::iterator Vector< T >::end () [noexcept]
```

end funkcija

Definition at line 184 of file [vector.hpp](#).

Here is the caller graph for this function:



3.1.4.22 erase() [1/3]

```
template<typename T>
Vector< T >::iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last)
```

erase funkcija intervalui

Definition at line 442 of file [vector.hpp](#).

Here is the call graph for this function:

**3.1.4.23 erase()** [2/3]

```
template<typename T>
Vector< T >::iterator Vector< T >::erase (
    const_iterator pos)
```

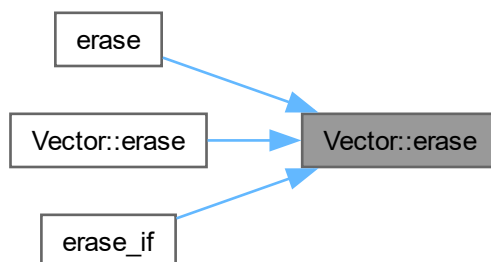
erase funkcija su vienu iteratoriumi

Definition at line 431 of file [vector.hpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:

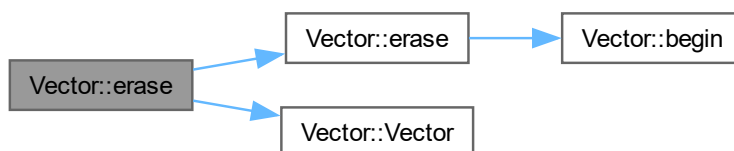


3.1.4.24 erase() [3/3]

```

template<typename T>
void Vector< T >::erase (
    Vector< T > & v,
    const T & value)
  
```

Here is the call graph for this function:



3.1.4.25 erase_if()

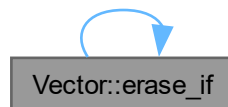
```

template<typename T>
template<typename Pred>
void Vector< T >::erase_if (
    Vector< T > v,
    Pred p)
  
```

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.4.26 front()

```
template<typename T>
Vector< T >::reference Vector< T >::front ()
```

front funkcija

Definition at line 154 of file [vector.hpp](#).

3.1.4.27 get_allocator()

```
template<typename T>
Vector< T >::allocator_type Vector< T >::get_allocator () const
```

get allocator funkcija

Definition at line 136 of file [vector.hpp](#).

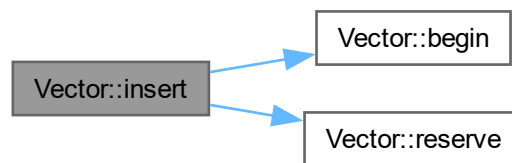
3.1.4.28 insert() [1/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    const T & value)
```

insert funkcija su const T& value

Definition at line 284 of file [vector.hpp](#).

Here is the call graph for this function:



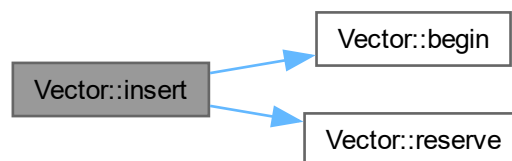
3.1.4.29 insert() [2/5]

```
template<typename T>
template<typename InputIt, typename>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last)
```

insert funkcija su iteratoriais

Definition at line 336 of file [vector.hpp](#).

Here is the call graph for this function:



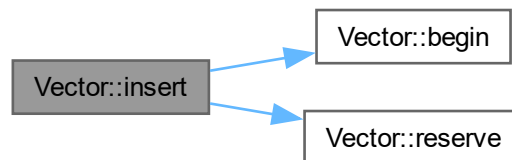
3.1.4.30 insert() [3/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    size_type count,
    const T & value)
```

insert funkcija su count ir value

Definition at line 316 of file [vector.hpp](#).

Here is the call graph for this function:



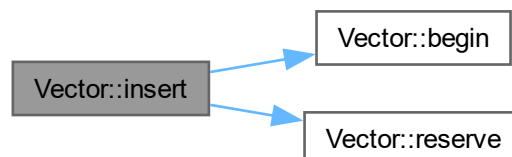
3.1.4.31 insert() [4/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    std::initializer_list< T > ilist)
```

insert funkcija su initializer list

Definition at line 358 of file [vector.hpp](#).

Here is the call graph for this function:



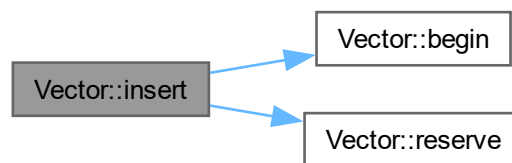
3.1.4.32 insert() [5/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    T && value)
```

insert funkcija su T&& value

Definition at line 300 of file [vector.hpp](#).

Here is the call graph for this function:



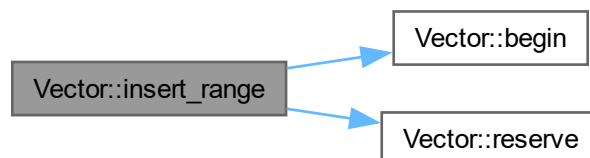
3.1.4.33 insert_range()

```
template<typename T>
Vector< T >::iterator Vector< T >::insert_range (
    const_iterator pos,
    std::initializer_list< T > ilist) [constexpr]
```

insert range funkcija su initializer list

Definition at line 380 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.34 max_size()

```
template<typename T>
Vector< T >::size_type Vector< T >::max_size () const [noexcept]
```

max size funkcija

Definition at line 239 of file [vector.hpp](#).

3.1.4.35 operator!=(())

```
template<typename T>
bool Vector< T >::operator!= (
    const Vector< T > & other) const
```

operator !=

Definition at line 549 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.36 operator<()

```
template<typename T>
bool Vector< T >::operator< (
    const Vector< T > & other) const
```

operator <

Definition at line 554 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.37 operator<=()

```
template<typename T>
bool Vector< T >::operator<= (
    const Vector< T > & other) const
```

operator <=

Definition at line 576 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.38 operator=() [1/2]

```
template<typename T>
Vector< T > & Vector< T >::operator= (
    const Vector< T > & r)
```

kopijavimo konstruktorius

kopijavimo priskyrimo operatorius

Definition at line 56 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.39 `operator=()` [2/2]

```
template<typename T>
Vector< T > & Vector< T >::operator= (
    Vector< T > && r)
```

perkelimo(move) konstruktorius

move priskyrimo operatorius

Definition at line 71 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.40 `operator==()`

```
template<typename T>
bool Vector< T >::operator== (
    const Vector< T > & other) const
```

non-member functions

operator ==

Definition at line 533 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.41 operator>()

```
template<typename T>
bool Vector< T >::operator> (
    const Vector< T > & other) const
```

operator >

Definition at line 571 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.42 operator>=()

```
template<typename T>
bool Vector< T >::operator>= (
    const Vector< T > & other) const
```

operator >=

Definition at line 581 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.43 operator[]()

```
template<typename T>
Vector< T >::reference Vector< T >::operator[] (
    size_type pos)
```

operator []

Definition at line 149 of file [vector.hpp](#).

3.1.4.44 pop_back()

```
template<typename T>  
void Vector< T >::pop_back ()
```

pop back funkcija

Definition at line 484 of file [vector.hpp](#).

3.1.4.45 push_back() [1/2]

```
template<typename T>  
void Vector< T >::push_back (  
    const T & value)
```

push back funkcija su const T& value

Definition at line 454 of file [vector.hpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



3.1.4.46 `push_back()` [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value)
```

push back funkcija su T&& value

Definition at line 464 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.47 `rbegin()` [1/2]

```
template<typename T>
Vector< T >::const_reverse_iterator Vector< T >::rbegin () const [noexcept]
```

const rbegin funkcija

Definition at line 204 of file [vector.hpp](#).

Here is the call graph for this function:



3.1.4.48 rbegin() [2/2]

```
template<typename T>  
Vector< T >::reverse_iterator Vector< T >::rbegin () [noexcept]
```

rbegin funkcija

Definition at line 199 of file [vector.hpp](#).

Here is the call graph for this function:

**3.1.4.49 rend()** [1/2]

```
template<typename T>  
Vector< T >::const_reverse_iterator Vector< T >::rend () const [noexcept]
```

const rend funkcija

Definition at line 219 of file [vector.hpp](#).

Here is the call graph for this function:

**3.1.4.50 rend()** [2/2]

```
template<typename T>  
Vector< T >::reverse_iterator Vector< T >::rend () [noexcept]
```

rend funkcija

Definition at line 214 of file [vector.hpp](#).

Here is the call graph for this function:



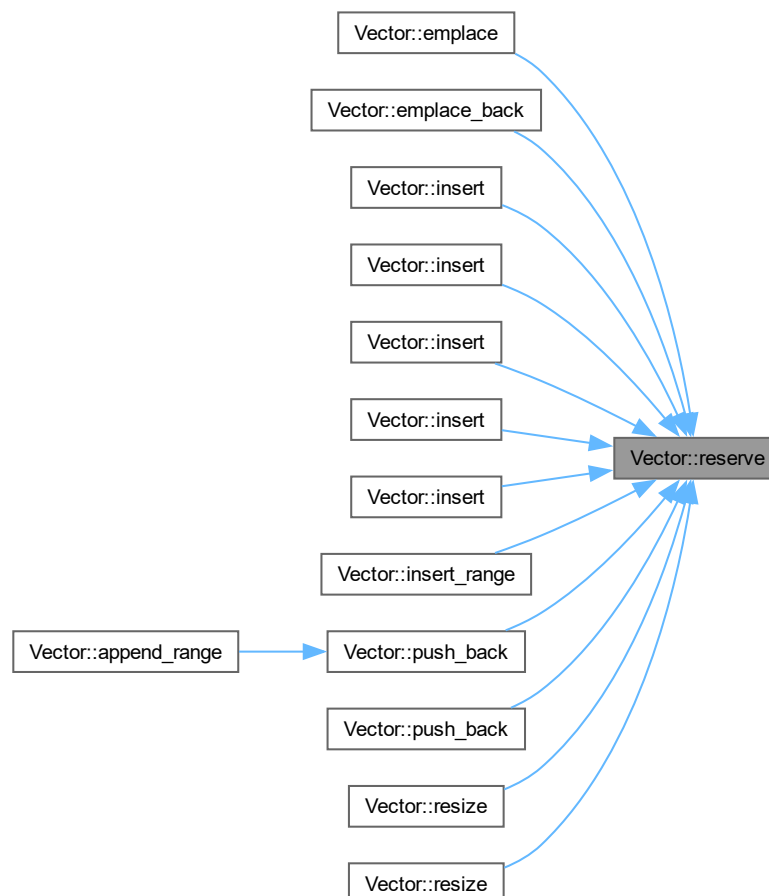
3.1.4.51 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_type new_cap)
```

reserve funkcija

Definition at line 249 of file [vector.hpp](#).

Here is the caller graph for this function:



3.1.4.52 `resize()` [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count)
```

`resize` funkcija su vienu parametru

Definition at line 492 of file [vector.hpp](#).

Here is the call graph for this function:

**3.1.4.53** `resize()` [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count,
    const value_type & value)
```

`resize` funkcija su dvie parametrais

Definition at line 509 of file [vector.hpp](#).

Here is the call graph for this function:

**3.1.4.54** `shrink_to_fit()`

```
template<typename T>
void Vector< T >::shrink_to_fit ()
```

`shrink to fit` funkcija

Definition at line 264 of file [vector.hpp](#).

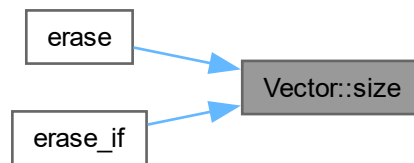
3.1.4.55 size()

```
template<typename T>
Vector< T >::size_type Vector< T >::size () const [noexcept]
```

size funkcija

Definition at line 234 of file [vector.hpp](#).

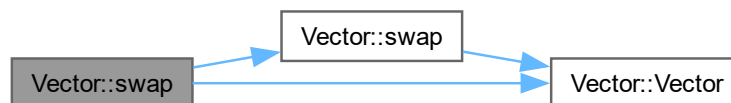
Here is the caller graph for this function:



3.1.4.56 swap() [1/2]

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & a,
    Vector< T > & b) [noexcept]
```

Here is the call graph for this function:



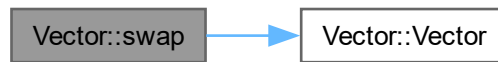
3.1.4.57 swap() [2/2]

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [constexpr], [noexcept]
```

swap funkcija

Definition at line 526 of file [vector.hpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following files:

- [vector.h](#)
- [vector.hpp](#)

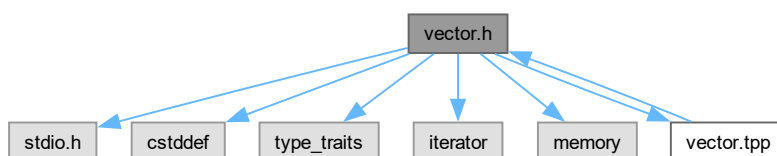
Chapter 4

File Documentation

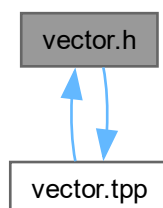
4.1 vector.h File Reference

```
#include <stdio.h>
#include <cstdint>
#include <type_traits>
#include <iterator>
#include <memory>
#include "vector.hpp"
```

Include dependency graph for vector.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Vector< T >](#)

4.2 vector.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <stdio.h>
00003 #include <cstdint>
00004 #include <type_traits>
00005 #include <iterator>
00006 #include <memory>
00007
00008 template <typename T>
00009 class Vector {
00010 public:
00011     //member types
00012     using value_type = T;
00013     using allocator_type = std::allocator<T>;
00014     using size_type = std::size_t;
00015     using difference_type = std::ptrdiff_t;
00016     using reference = value_type&;
00017     using const_reference = const value_type&;
00018     using pointer = value_type*;
00019     using const_pointer = const value_type*;
00020     using iterator = value_type*;
00021     using const_iterator = const value_type*;
00022     using reverse_iterator = std::reverse_iterator<iterator>;
00023     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00024 private:
00025     T* elem;
00026     size_type sz;
00027     size_type cap;
00028 public:
00029     Vector();
00030     Vector(size_type s);
00031     Vector(std::initializer_list<T> init);
00032     Vector(const Vector& r);
00033     Vector& operator=(const Vector& r);
00034     Vector(Vector&& r);
00035     Vector& operator=(Vector&& r);
00036     ~Vector();
00037
00038     void assign(size_type count, const T& value);
00039     void assign(std::initializer_list<T> ilist);
00040     template <typename InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>
00041     void assign(InputIt first, InputIt last);
00042     template< typename R>
00043     constexpr void assign_range(R&& r);
00044     allocator_type get_allocator() const;
00045     reference at(size_type pos);
00046     reference operator[](size_type pos);
00047     reference front();
00048     reference back();
00049     T* data() noexcept;
00050     iterator begin() noexcept;
00051     const_iterator begin() const noexcept;
00052     const_iterator cbegin() const noexcept;
00053     iterator end() noexcept;
00054     const_iterator end() const noexcept;
00055     const_iterator cend() const noexcept;
00056     reverse_iterator rbegin() noexcept;
00057     const_reverse_iterator rbegin() const noexcept;
00058     const_reverse_iterator crbegin() const noexcept;
00059     reverse_iterator rend() noexcept;
00060     const_reverse_iterator rend() const noexcept;
00061     const_reverse_iterator crend() const noexcept;
00062     bool empty() const noexcept;
00063     size_type size() const noexcept;
00064     size_type max_size() const noexcept;
00065     size_type capacity() const noexcept;
00066     void reserve(size_type new_cap);
00067     void shrink_to_fit();
00068     void clear() noexcept;
00069     iterator insert(const_iterator pos, const T& value);
00070     iterator insert(const_iterator pos, T&& value);
00071     iterator insert(const_iterator pos, size_type count, const T& value);
00072     template <typename InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>
00073     iterator insert(const_iterator pos, InputIt first, InputIt last);

```

```

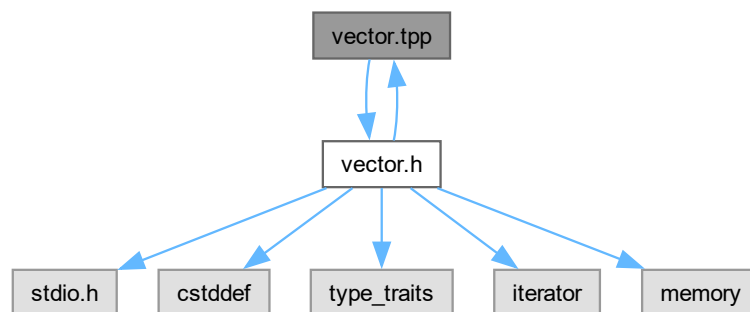
00080     iterator insert(const_iterator pos, std::initializer_list<T> ilist);
00081
00082     constexpr iterator insert_range( const_iterator pos, std::initializer_list<T> ilist);
00083     template <class... Args>
00084     iterator emplace(const_iterator pos, Args&&... args);
00085     template <typename... Args>
00086     reference emplace_back(Args&&... args);
00087     iterator erase(const_iterator pos);
00088     iterator erase(const_iterator first, const_iterator last);
00089     void push_back(const T& value);
00090     void push_back(T&& value);
00091     template <typename R>
00092     constexpr void append_range(R&& rg);
00093     void pop_back();
00094     void resize(size_type count);
00095     void resize(size_type count, const value_type& value);
00096     constexpr void swap(Vector& other) noexcept;
00097     bool operator==(const Vector& other) const;
00098     bool operator!=(const Vector& other) const;
00099     bool operator<(const Vector& other) const;
00100     bool operator<=(const Vector& other) const;
00101     bool operator>(const Vector& other) const;
00102     bool operator>=(const Vector& other) const;
00103     void swap(Vector& a, Vector& b) noexcept;
00104     void erase(Vector& v, const T& value);
00105     template <typename Pred>
00106     void erase_if(Vector v, Pred p);
00107 };
00108 #include "vector.hpp"

```

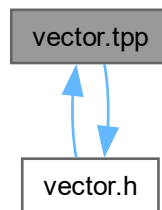
4.3 vector.hpp File Reference

```
#include "vector.h"
```

Include dependency graph for vector.hpp:



This graph shows which files directly or indirectly include this file:



Functions

- `template<typename T>`
`void swap (Vector< T > &a, Vector< T > &b) noexcept`
swap funkcija friend
- `template<typename T>`
`void erase (Vector< T > &v, const T &value)`
erase funkcija su verte
- `template<typename T, typename Pred>`
`void erase_if (Vector< T > &v, Pred p)`
erase if funkcija

4.3.1 Function Documentation

4.3.1.1 erase()

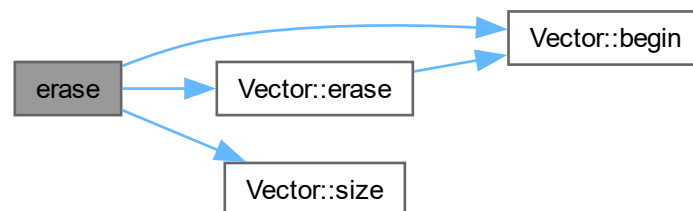
```

template<typename T>
void erase (
    Vector< T > & v,
    const T & value)
  
```

erase funkcija su verte

Definition at line 591 of file [vector.hpp](#).

Here is the call graph for this function:



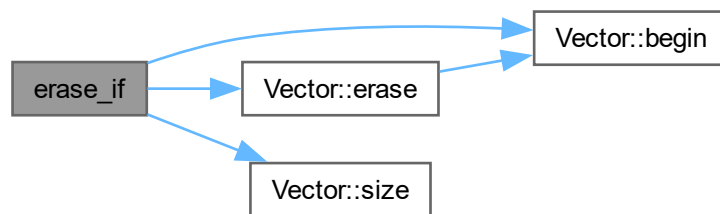
4.3.1.2 erase_if()

```
template<typename T, typename Pred>
void erase_if (
    Vector< T > & v,
    Pred p)
```

erase if funkcija

Definition at line 603 of file [vector.hpp](#).

Here is the call graph for this function:



4.3.1.3 swap()

```
template<typename T>
void swap (
    Vector< T > & a,
    Vector< T > & b) [noexcept]
```

swap funkcija friend

Definition at line 586 of file [vector.hpp](#).

4.4 vector.hpp

[Go to the documentation of this file.](#)

```
00001 #include "vector.h"
00002
00003 template <typename T>
00004 Vector<T>::Vector()
00005 {
00006     elem = nullptr;
00007     sz = 0;
00008     cap = 0;
00009 }
00010 template <typename T>
00011 Vector<T>::Vector(size_type s)
00012 {
00013     elem = new T[s];
00014     sz = s;
00015     cap = s;
```

```

00016 }
00017 template <typename T>
00018 Vector<T>::Vector(std::initializer_list<T> init)
00019 {
00020     sz = init.size();
00021     cap = init.size();
00022     elem = (cap == 0) ? nullptr : new T[cap];
00023     size_type i = 0;
00024     for(const auto& value : init){
00025         elem[i++] = value;
00026     }
00027 }
00028 template <typename T>
00029 Vector<T>::~Vector()//destruktorius
00030 {
00031     delete[] elem;
00032     sz = 0;
00033     cap = 0;
00034 }
00035 template <typename T>
00036 Vector<T>::Vector(const Vector& r)
00037 {
00038     elem = new T[r.cap];
00039     sz = r.sz;
00040     cap = r.cap;
00041     for(size_type i = 0; i < sz; i++){
00042         elem[i] = r.elem[i];
00043     }
00044 }
00045 template <typename T>
00046 Vector<T>::Vector(Vector&& r)
00047 {
00048     elem = r.elem;
00049     sz = r.sz;
00050     cap = r.cap;
00051     r.elem = nullptr;
00052     r.sz = 0;
00053     r.cap = 0;
00054 }
00055 template <typename T>
00056 Vector<T>& Vector<T>::operator=(const Vector& r)
00057 {
00058     if(this == &r)
00059         return *this;
00060     delete[] elem;
00061     sz = r.sz;
00062     cap = r.cap;
00063     elem = new T[cap];
00064     for(size_type i = 0; i < sz; i++){
00065         elem[i] = r.elem[i];
00066     }
00067     return *this;
00068 }
00069 }
00070 template <typename T>
00071 Vector<T>& Vector<T>::operator=(Vector&& r)
00072 {
00073     if(this ==&r)
00074         return *this;
00075     delete[] elem;
00076     elem = r.elem;
00077     sz = r.sz;
00078     cap = r.cap;
00079     r.elem = nullptr;
00080     r.sz=0;
00081     r.cap = 0;
00082     return *this;
00083 }
00084 }
00085 }
00086 template <typename T>
00087 void Vector<T>::assign(size_type count, const T& value)
00088 {
00089     delete[] elem;
00090     elem = new T[count];
00091     sz = count;
00092     cap = count;
00093     for(size_type i = 0; i < sz; i++){
00094         elem[i] = value;
00095     }
00096 }
00097 template <typename T>
00098 void Vector<T>::assign(std::initializer_list<T> ilist)
00099 {
00100     delete[] elem;
00101     sz = ilist.size();
00102     cap = ilist.size();

```

```

00103     elem = (cap == 0) ? nullptr : new T[cap];
00104     size_type i = 0;
00105     for(const auto& value : ilist){
00106         elem[i++] = value;
00107     }
00108 }
00109 template <typename T>
00110 template <typename InputIt, typename>
00111 void Vector<T>::assign(InputIt first, InputIt last)
00112 {
00113     delete[] elem;
00114     sz = static_cast<size_type>(std::distance(first, last));
00115     cap = sz;
00116     elem = (cap == 0) ? nullptr : new T[cap];
00117     size_type i = 0;
00118     for(InputIt it = first; it != last; ++it){
00119         elem[i++] = *it;
00120     }
00121 }
00122 template <typename T>
00123 template <typename R>
00124 constexpr void Vector<T>::assign_range(R&& r)
00125 {
00126     delete[] elem;
00127     sz = static_cast<size_type>(std::distance(std::begin(r), std::end(r)));
00128     cap = sz;
00129     elem = (cap == 0) ? nullptr : new T[cap];
00130     size_type i = 0;
00131     for(auto&& value : r){
00132         elem[i++] = std::forward<decltype(value)>(value);
00133     }
00134 }
00135 template <typename T>
00136 typename Vector<T>::allocator_type Vector<T>::get_allocator() const
00137 {
00138     return allocator_type();
00139 }
00140 template <typename T>
00141 typename Vector<T>::reference Vector<T>::at(size_type pos)
00142 {
00143     if((pos < 0) || (pos >= sz)){
00144         throw std::out_of_range("Index out of range");
00145     }
00146     return elem[pos];
00147 }
00148 template <typename T>
00149 typename Vector<T>::reference Vector<T>::operator[](size_type pos)
00150 {
00151     return elem[pos];
00152 }
00153 template <typename T>
00154 typename Vector<T>::reference Vector<T>::front()
00155 {
00156     return elem[0];
00157 }
00158 template <typename T>
00159 typename Vector<T>::reference Vector<T>::back()
00160 {
00161     return elem[sz - 1];
00162 }
00163 template <typename T>
00164 T* Vector<T>::data() noexcept
00165 {
00166     return elem;
00167 }
00168 template <typename T>
00169 typename Vector<T>::iterator Vector<T>::begin() noexcept
00170 {
00171     return elem;
00172 }
00173 template <typename T>
00174 typename Vector<T>::const_iterator Vector<T>::begin() const noexcept//const begin funkcija
00175 {
00176     return elem;
00177 }
00178 template <typename T>
00179 typename Vector<T>::const_iterator Vector<T>::cbegin() const noexcept
00180 {
00181     return elem;
00182 }
00183 template <typename T>
00184 typename Vector<T>::iterator Vector<T>::end() noexcept
00185 {
00186     return elem + sz;
00187 }
00188 template <typename T>
00189 typename Vector<T>::const_iterator Vector<T>::end() const noexcept

```

```

00190 {
00191     return elem + sz;
00192 }
00193 template <typename T>
00194 typename Vector<T>::const_iterator Vector<T>::cend() const noexcept
00195 {
00196     return elem + sz;
00197 }
00198 template <typename T>
00199 typename Vector<T>::reverse_iterator Vector<T>::rbegin() noexcept
00200 {
00201     return reverse_iterator(end());
00202 }
00203 template <typename T>
00204 typename Vector<T>::const_reverse_iterator Vector<T>::rbegin() const noexcept
00205 {
00206     return const_reverse_iterator(end());
00207 }
00208 template <typename T>
00209 typename Vector<T>::const_reverse_iterator Vector<T>::crbegin() const noexcept
00210 {
00211     return const_reverse_iterator(cend());
00212 }
00213 template <typename T>
00214 typename Vector<T>::reverse_iterator Vector<T>::rend() noexcept
00215 {
00216     return reverse_iterator(begin());
00217 }
00218 template <typename T>
00219 typename Vector<T>::const_reverse_iterator Vector<T>::rend() const noexcept
00220 {
00221     return const_reverse_iterator(begin());
00222 }
00223 template <typename T>
00224 typename Vector<T>::const_reverse_iterator Vector<T>::crend() const noexcept
00225 {
00226     return const_reverse_iterator(cbegin());
00227 }
00228 template <typename T>
00229 bool Vector<T>::empty() const noexcept
00230 {
00231     return sz == 0;
00232 }
00233 template <typename T>
00234 typename Vector<T>::size_type Vector<T>::size() const noexcept
00235 {
00236     return sz;
00237 }
00238 template <typename T>
00239 typename Vector<T>::size_type Vector<T>::max_size() const noexcept
00240 {
00241     return std::allocator_traits<allocator_type>::max_size(allocator_type());
00242 }
00243 template <typename T>
00244 typename Vector<T>::size_type Vector<T>::capacity() const noexcept
00245 {
00246     return cap;
00247 }
00248 template <typename T>
00249 void Vector<T>::reserve(size_type new_cap)
00250 {
00251     if(new_cap > cap)
00252     {
00253         T* new_elem = new T[new_cap];
00254         for(size_type i = 0; i < sz; i++)
00255         {
00256             new_elem[i] = elem[i];
00257         }
00258         delete[] elem;
00259         elem = new_elem;
00260         cap = new_cap;
00261     }
00262 }
00263 template <typename T>
00264 void Vector<T>::shrink_to_fit()
00265 {
00266     if(sz < cap)
00267     {
00268         T* new_elem = (sz == 0) ? nullptr : new T[sz];
00269         for(size_type i = 0; i < sz; i++)
00270         {
00271             new_elem[i] = elem[i];
00272         }
00273         delete[] elem;
00274         elem = new_elem;
00275         cap = sz;
00276     }

```

```

00277 }
00278 template <typename T>
00279 void Vector<T>::clear() noexcept
00280 {
00281     sz = 0;
00282 }
00283 template <typename T>
00284 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, const T& value)
00285 {
00286     size_type index = pos - begin();
00287     if (sz >= cap)
00288     {
00289         reserve(cap == 0 ? 1 : cap * 2);
00290     }
00291     for (size_type i = sz; i > index; i--)
00292     {
00293         elem[i] = elem[i - 1];
00294     }
00295     elem[index] = value;
00296     ++sz;
00297     return begin() + index;
00298 }
00299 template <typename T>
00300 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, T&& value)
00301 {
00302     size_type index = pos - begin();
00303     if (sz >= cap)
00304     {
00305         reserve(cap == 0 ? 1 : cap * 2);
00306     }
00307     for (size_type i = sz; i > index; i--)
00308     {
00309         elem[i] = std::move(elem[i - 1]);
00310     }
00311     elem[index] = std::move(value);
00312     ++sz;
00313     return begin() + index;
00314 }
00315 template <typename T>
00316 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, size_type count, const T& value)
00317 {
00318     size_type index = pos - begin();
00319     if (sz + count > cap)
00320     {
00321         reserve((cap == 0) ? count : std::max(cap * 2, sz + count));
00322     }
00323     for (size_type i = sz; i >= index; i--)
00324     {
00325         elem[i + count - 1] = elem[i - 1];
00326     }
00327     for (size_type i = 0; i < count; i++)
00328     {
00329         elem[index + i] = value;
00330     }
00331     sz += count;
00332     return begin() + index;
00333 }
00334 template <typename T>
00335 template <typename InputIt, typename>
00336 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, InputIt first, InputIt last)
00337 {
00338     size_type index = pos - begin();
00339     size_type count = static_cast<size_type>(std::distance(first, last));
00340     if (sz + count > cap)
00341     {
00342         reserve((cap == 0) ? count : std::max(cap * 2, sz + count));
00343     }
00344     for (size_type i = sz; i >= index; i--)
00345     {
00346         elem[i + count - 1] = elem[i - 1];
00347     }
00348     size_type i = 0;
00349     for (InputIt it = first; it != last; ++it)
00350     {
00351         elem[index + i] = *it;
00352         ++i;
00353     }
00354     sz += count;
00355     return begin() + index;
00356 }
00357 template <typename T>
00358 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, std::initializer_list<T> ilist)
00359 {
00360     size_type index = pos - begin();
00361     size_type count = ilist.size();
00362     if (sz + count > cap)
00363     {

```

```

00364         reserve((cap == 0) ? count : std::max(cap * 2, sz + count));
00365     }
00366     for(size_type i = sz; i >= index; i--)
00367     {
00368         elem[i + count - 1] = elem[i - 1];
00369     }
00370     size_type i = 0;
00371     for(const auto& value : ilist)
00372     {
00373         elem[index + i] = value;
00374         ++i;
00375     }
00376     sz += count;
00377     return begin() + index;
00378 }
00379 template <typename T>
00380 constexpr typename Vector<T>::iterator Vector<T>::insert_range(const_iterator pos,
00381     std::initializer_list<T> ilist)
00382 {
00383     size_type index = pos - begin();
00384     size_type count = ilist.size();
00385     if(sz + count > cap)
00386     {
00387         reserve((cap == 0) ? count : std::max(cap * 2, sz + count));
00388     }
00389     for(size_type i = sz; i >= index; i--)
00390     {
00391         elem[i + count - 1] = elem[i - 1];
00392     }
00393     size_type i = 0;
00394     for(const auto& value : ilist)
00395     {
00396         elem[index + i] = value;
00397         ++i;
00398     }
00399     sz += count;
00400     return begin() + index;
00401 }
00402 template <typename T>
00403 template <class... Args>
00404 typename Vector<T>::iterator Vector<T>::emplace(const_iterator pos, Args&&... args)
00405 {
00406     size_type index = pos - begin();
00407     if(sz >= cap)
00408     {
00409         reserve(cap == 0 ? 1 : cap * 2);
00410     }
00411     for(size_type i = sz; i > index; i--)
00412     {
00413         elem[i] = std::move(elem[i - 1]);
00414     }
00415     elem[index] = T(std::forward<Args>(args)...);
00416     ++sz;
00417     return begin() + index;
00418 }
00419 template <typename T>
00420 template <typename... Args>
00421 typename Vector<T>::reference Vector<T>::emplace_back(Args&&... args)
00422 {
00423     if(sz >= cap)
00424     {
00425         reserve(cap == 0 ? 1 : cap * 2);
00426     }
00427     elem[sz] = T(std::forward<Args>(args)...);
00428     ++sz;
00429     return elem[sz - 1];
00430 }
00431 template <typename T>
00432 typename Vector<T>::iterator Vector<T>::erase(const_iterator pos)
00433 {
00434     size_type index = pos - begin();
00435     for(size_type i = index; i < sz - 1; i++)
00436     {
00437         elem[i] = std::move(elem[i + 1]);
00438     }
00439     --sz;
00440     return begin() + index;
00441 }
00442 template <typename T>
00443 template <typename... Args>
00444 typename Vector<T>::reference Vector<T>::erase(const_iterator first, const_iterator last)
00445 {
00446     size_type index = first - begin();
00447     size_type count = last - first;
00448     for(size_type i = index; i + count < sz; i++)
00449     {
00450         elem[i] = std::move(elem[i + count]);
00451     }

```

```

00450     sz -= count;
00451     return begin() + index;
00452 }
00453 template <typename T>
00454 void Vector<T>::push_back(const T& value)
00455 {
00456     if (sz >= cap)
00457     {
00458         reserve(cap == 0 ? 1 : cap * 2);
00459     }
00460     elem[sz] = value;
00461     ++sz;
00462 }
00463 template <typename T>
00464 void Vector<T>::push_back(T&& value)
00465 {
00466     if (sz >= cap)
00467     {
00468         reserve(cap == 0 ? 1 : cap * 2);
00469     }
00470     elem[sz] = std::move(value);
00471     ++sz;
00472 }
00473 template <typename T>
00474 template <typename R>
00475 constexpr void Vector<T>::append_range(R&& rg)
00476 {
00477     for (auto&& value : rg)
00478     {
00479         push_back(value);
00480     }
00481 }
00482 }
00483 template <typename T>
00484 void Vector<T>::pop_back()
00485 {
00486     if (sz > 0)
00487     {
00488         --sz;
00489     }
00490 }
00491 template <typename T>
00492 void Vector<T>::resize(size_type count)
00493 {
00494     if (count < sz)
00495     {
00496         sz = count;
00497     }
00498     else if (count > sz)
00499     {
00500         reserve(count);
00501         for (size_type i = sz; i < count; i++)
00502         {
00503             elem[i] = T();
00504         }
00505         sz = count;
00506     }
00507 }
00508 template <typename T>
00509 void Vector<T>::resize(size_type count, const value_type& value)
00510 {
00511     if (count < sz)
00512     {
00513         sz = count;
00514     }
00515     else if (count > sz)
00516     {
00517         reserve(count);
00518         for (size_type i = sz; i < count; i++)
00519         {
00520             elem[i] = value;
00521         }
00522         sz = count;
00523     }
00524 }
00525 template <typename T>
00526 constexpr void Vector<T>::swap(Vector<T>& other) noexcept
00527 {
00528     std::swap(elem, other.elem);
00529     std::swap(sz, other.sz);
00530     std::swap(cap, other.cap);
00531 }
00532 template <typename T>
00533 bool Vector<T>::operator==(const Vector& other) const
00534 {
00535     if (sz != other.sz)
00536     {

```

```

00537         return false;
00538     }
00539     for(size_type i = 0; i < sz; i++)
00540     {
00541         if(elem[i] != other.elem[i])
00542         {
00543             return false;
00544         }
00545     }
00546     return true;
00547 }
00548 template <typename T>
00549 bool Vector<T>::operator!=(const Vector& other) const
00550 {
00551     return !(*this == other);
00552 }
00553 template <typename T>
00554 bool Vector<T>::operator<(const Vector& other) const
00555 {
00556     size_type min_size = std::min(sz, other.sz);
00557     for(size_type i = 0; i < min_size; i++)
00558     {
00559         if(elem[i] < other.elem[i])
00560         {
00561             return true;
00562         }
00563         else if(elem[i] > other.elem[i])
00564         {
00565             return false;
00566         }
00567     }
00568     return sz < other.sz;
00569 }
00570 template <typename T>
00571 bool Vector<T>::operator>(const Vector& other) const
00572 {
00573     return other < *this;
00574 }
00575 template <typename T>
00576 bool Vector<T>::operator<=(const Vector& other) const
00577 {
00578     return !(other < *this);
00579 }
00580 template <typename T>
00581 bool Vector<T>::operator>=(const Vector& other) const
00582 {
00583     return !(*this < other);
00584 }
00585 template <typename T>
00586 void swap(Vector<T>& a, Vector<T>& b) noexcept
00587 {
00588     a.swap(b);
00589 }
00590 template <typename T>
00591 void erase(Vector<T>& v, const T& value)
00592 {
00593     for(typename Vector<T>::size_type i = 0; i < v.size(); i++)
00594     {
00595         if(v[i] == value)
00596         {
00597             v.erase(v.begin() + i);
00598             --i;
00599         }
00600     }
00601 }
00602 template <typename T, typename Pred>
00603 void erase_if(Vector<T>& v, Pred p)
00604 {
00605     typename Vector<T>::size_type i = 0;
00606     while(i < v.size())
00607     {
00608         if(p(v[i]))
00609         {
00610             v.erase(v.begin() + i);
00611         }
00612         else{
00613             ++i;
00614         }
00615     }
00616 }
00617 }

```


Index

`~Vector`
 `Vector< T >`, [13](#)

`allocator_type`
 `Vector< T >`, [8](#)

`append_range`
 `Vector< T >`, [13](#)

`assign`
 `Vector< T >`, [13](#), [14](#)

`assign_range`
 `Vector< T >`, [14](#)

`at`
 `Vector< T >`, [14](#)

`back`
 `Vector< T >`, [15](#)

`begin`
 `Vector< T >`, [15](#)

`capacity`
 `Vector< T >`, [16](#)

`cbegin`
 `Vector< T >`, [16](#)

`cend`
 `Vector< T >`, [17](#)

`clear`
 `Vector< T >`, [17](#)

`const_iterator`
 `Vector< T >`, [8](#)

`const_pointer`
 `Vector< T >`, [8](#)

`const_reference`
 `Vector< T >`, [8](#)

`const_reverse_iterator`
 `Vector< T >`, [9](#)

`crbegin`
 `Vector< T >`, [17](#)

`crend`
 `Vector< T >`, [18](#)

`data`
 `Vector< T >`, [18](#)

`difference_type`
 `Vector< T >`, [9](#)

`emplace`
 `Vector< T >`, [18](#)

`emplace_back`
 `Vector< T >`, [19](#)

`empty`
 `Vector< T >`, [19](#)

`end`
 `Vector< T >`, [20](#)

`erase`
 `Vector< T >`, [20–22](#)
 `vector.hpp`, [42](#)

`erase_if`
 `Vector< T >`, [22](#)
 `vector.hpp`, [42](#)

`front`
 `Vector< T >`, [23](#)

`get_allocator`
 `Vector< T >`, [23](#)

`insert`
 `Vector< T >`, [23–25](#)

`insert_range`
 `Vector< T >`, [26](#)

`iterator`
 `Vector< T >`, [9](#)

`max_size`
 `Vector< T >`, [26](#)

`operator!=`
 `Vector< T >`, [27](#)

`operator<`
 `Vector< T >`, [27](#)

`operator<=`
 `Vector< T >`, [27](#)

`operator>`
 `Vector< T >`, [29](#)

`operator>=`
 `Vector< T >`, [30](#)

`operator=`
 `Vector< T >`, [28](#)

`operator==`
 `Vector< T >`, [29](#)

`operator[]`
 `Vector< T >`, [30](#)

`pointer`
 `Vector< T >`, [9](#)

`pop_back`
 `Vector< T >`, [30](#)

`push_back`
 `Vector< T >`, [31](#)

`rbegin`
 `Vector< T >`, [32](#)

- reference
 - Vector< T >, 9
- rend
 - Vector< T >, 33
- reserve
 - Vector< T >, 34
- resize
 - Vector< T >, 34, 35
- reverse_iterator
 - Vector< T >, 9
- shrink_to_fit
 - Vector< T >, 35
- size
 - Vector< T >, 35
- size_type
 - Vector< T >, 9
- swap
 - Vector< T >, 36
 - vector.hpp, 43
- value_type
 - Vector< T >, 10
- Vector
 - Vector< T >, 10–12
- Vector< T >, 5
 - ~Vector, 13
 - allocator_type, 8
 - append_range, 13
 - assign, 13, 14
 - assign_range, 14
 - at, 14
 - back, 15
 - begin, 15
 - capacity, 16
 - cbegin, 16
 - cend, 17
 - clear, 17
 - const_iterator, 8
 - const_pointer, 8
 - const_reference, 8
 - const_reverse_iterator, 9
 - crbegin, 17
 - crend, 18
 - data, 18
 - difference_type, 9
 - emplace, 18
 - emplace_back, 19
 - empty, 19
 - end, 20
 - erase, 20–22
 - erase_if, 22
 - front, 23
 - get_allocator, 23
 - insert, 23–25
 - insert_range, 26
 - iterator, 9
 - max_size, 26
 - operator!=, 27
 - operator<, 27
 - operator<=, 27
 - operator>, 29
 - operator>=, 30
 - operator=, 28
 - operator==, 29
 - operator[], 30
 - pointer, 9
 - pop_back, 30
 - push_back, 31
 - rbegin, 32
 - reference, 9
 - rend, 33
 - reserve, 34
 - resize, 34, 35
 - reverse_iterator, 9
 - shrink_to_fit, 35
 - size, 35
 - size_type, 9
 - swap, 36
 - value_type, 10
 - Vector, 10–12
- vector.h, 39
- vector.hpp, 41
 - erase, 42
 - erase_if, 42
 - swap, 43